

Use Phantocite in eduGenAI 2

Give an eduGenAI 2 agent an Action that verifies a paper's references against OpenAlex — inside your own chat, with no API key of your own.

eduGenAI 1 was withdrawn after vulnerabilities were found in an audit, and its Extension builder went with it. eduGenAI 2 (edugenai2.npuls.nl, sign in with SRAM) is built on LibreChat, where an assistant gains a tool from an OpenAPI schema instead of a hand-written function definition. These steps are for that.

How it works

- **The agent** reads the paper, extracts the reference list and the sentences that cite each source, judges whether each citation matches the source, and writes the report.
- **This action's endpoint** looks each reference up in OpenAlex and checks title, authors, year, journal, DOI, volume, issue and pages — deterministically, with no LLM — and returns the abstract.

Because the reasoning stays on eduGenAI's side, the action needs no LLM API key. It only wraps OpenAlex, which is free.

Before you start

- An eduGenAI 2 account (edugenai2.npuls.nl, SRAM / SURFconext).
- Agents with Actions enabled for your institution. Both are administrator settings in LibreChat, and reachable domains can be restricted to a whitelist. If the Agent Builder is missing or an action will not save, ask your institution's contact or edugenai@npuls.nl to enable agent actions and to allow www.phantocite.com.
- A model that supports tool calling — the GPT models on the platform do.

Step 1 — Create the agent

Open the Agent Builder in the sidebar, create an agent, give it a name and pick a model.

Name

Citation checker (OpenAlex)

Step 2 — Paste the instructions

Into the agent's Instructions field, verbatim. This is the whole workflow: what to extract, when to call the tool, how to report.

You check the reference list of a paper the user uploads. You have one tool, `verify_references`, which looks references up in OpenAlex.

WHEN TO RUN. When the user uploads a document and gives any go-ahead ("check this", "start", "verify the references", or just the file with no other instruction), run the four steps below end to end without asking a clarifying question first. The document in the conversation IS the input; never ask the user to paste the bibliography. Only if no document has been provided at all, ask for one, then run.

STEP 1 - EXTRACT (do this yourself, before calling the tool). Read the document. For every entry in the reference list, take: title, every author name in order, whether it ends in "et al.", year, DOI (only if printed), journal or venue, volume, issue, and pages. Also find the sentence in the body where each source is cited, with a sentence of context on either side - keep those in your notes, they are for step 3 and are never sent to the tool. Finally, list every in-text citation that has NO entry in the reference list; these ORPHAN CITATIONS cannot be looked up by any reader. If the paper has no reference list at all, every citation is an orphan: skip step 2 and report them all as missing.

STEP 2 - VERIFY. Call `verify_references` ONCE, passing every reference that has at least a title. Never call it with an empty or placeholder list. For each reference it returns: status (found / fuzzy / not_found / lookup_failed), a badge, a severity from 0 to 100, mismatched_fields and minor_fields, a field_check comparing each printed detail with the real record, the matched work with its abstract, and - for fuzzy matches - a list of candidate works.

STEP 3 - JUDGE THE USE. For every reference the tool matched, compare the abstract it returned with the citing sentences you kept in step 1. Decide whether the claim the paper attributes to the source is plausibly supported by it. If the source is clearly about something else, or is used for a narrower or broader claim than it supports, that is a MISQUOTE - raise that reference's severity to at least 80. Judge only from the abstract, and say so when the abstract is too thin to tell.

STEP 4 - REPORT. Never show the user the raw JSON; it is a machine interface. Write one Markdown table, one row per reference, sorted by descending severity, with columns: # | Reference (short) | Verdict | What is wrong. Under the table, add a short "Details" section for every row that needs a look, naming the exact difference (printed vs record). If there are orphan citations, add a "Missing from the reference list" section listing them. End with one line: how many references were checked, how many are clean, how many need a look.

REPORT ONLY THIS. No APA or formatting critique, no style fixes, no suggested extra literature, no praise, no summary of the paper, no advice - unless the user asks for it in a later message. These ARE citation errors and must be reported: wrong author names, wrong author order, wrong year, journal, DOI or pages, in-text citations that contradict their reference entry, and citations missing from the reference list. Punctuation, capitalisation and italics are style - stay silent on those.

NEVER invent a DOI, an author, a year or a verdict the tool did not return. status "lookup_failed" means the reference could not be checked - say exactly that; it is not evidence of fabrication. status "not_found" means OpenAlex has no such work - report it as a probable fabrication, to be confirmed by hand. If the tool call fails or returns nothing, say so in one sentence and stop; do not fall back on your own memory of the literature.

Step 3 — Add the Action

Click Add Action. LibreChat builds the tool from an OpenAPI schema, and this app publishes one:

```
https://www.phantocite.com/openapi/edugenai.json
```

Paste that URL into the schema box (or open it and paste the JSON itself, if your build has no import-from-URL field). Set Authentication to None — the endpoint needs no key. Save the action, then save the agent.

The schema declares one operation, `verify_references`, and one server, `https://www.phantocite.com`. LibreChat checks that an action's domain matches that server URL, so leave it as published unless you are self-hosting — in which case use your own deployment's `/openapi/edugenai.json`.

Optional — OpenAlex Premium: for higher OpenAlex rate limits, set the action's authentication to an API key sent as the custom header `X-OpenAlex-Key`. It is used per request and never stored. The tool works fine without one.

Step 4 — Test it

Start a chat with the agent, attach a paper, and ask:

```
Check the references in the attached paper.
```

The agent should extract the references itself, call `verify_references` once, and answer with a table. LibreChat shows the tool call in the message, so you can confirm it ran.

Troubleshooting

- **No Agent Builder, or Add Action missing or refusing to save.** An administrator setting — see Before you start. Nothing on this side works around it.
- **Invalid schema or domain mismatch.** LibreChat checks the action's domain against the servers URL in the schema; import the published URL unchanged.
- **The agent never calls the tool.** Either the model has no tool support (switch to a GPT model) or the instructions were not saved. Naming the tool in the prompt forces it: "Use the `verify_references` tool to check the references in the attached paper."
- **Raw JSON in the chat.** The instructions were truncated or not saved; re-paste Step 2.
- **count: 0.** Every response carries `api_version`, and one with no results carries a hint describing what arrived (key names only, never content). No `api_version` at all means the action points at an old deployment.
- **lookup_failed.** A failed lookup, not a fabrication — usually a reference with no usable title.

To check the endpoint independently of eduGenAI, run this from any terminal — a healthy deployment answers in a couple of seconds with `count: 1` and a Verified result:

```
curl -s -X POST https://www.phantocite.com/api/verify_batch \  
-H "Content-Type: application/json" \  
-d '{"references":[{"title":"Attention is all you need","year":2017}]}'
```

Notes & limits

- No LLM key is stored or used by the action; OpenAlex is free and needs no key. The agent's own model runs on eduGenAI's side, under its terms.

- Only bibliographic metadata reaches this endpoint. The paper's full text and the citing sentences stay inside eduGenAI.
- OpenAlex's canonical year can differ from a printed year (online-first vs issue year) — treat a lone year mismatch as a prompt to double-check, not a verdict.
- The misquote judgement uses the abstract only, so a claim supported by the full text but not the abstract may read as uncertain. Treat results as leads for a human reviewer.
- Batch requests are capped at 200 references.